

Contents

Gradle — kurz erklärt	1
Warum überhaupt ein Build-Tool?	1
Was beim Kompilieren eigentlich passiert	1
Die drei wichtigsten Begriffe	2
Die drei Befehle, die ihr braucht	2
Was, wenn ein Befehl fehlschlägt?	3
In IntelliJ	3

Gradle — kurz erklärt

Dieses Projekt nutzt **Gradle** als Build-Tool. Ihr müsst Gradle selbst nicht konfigurieren — das ist bereits fertig eingerichtet (`build.gradle.kts`, `settings.gradle.kts`) — aber ein kurzer Überblick hilft, zu verstehen, was die drei Befehle tun, die ihr im Praktikum tatsächlich braucht.

Warum überhaupt ein Build-Tool?

Euer Projekt besteht nicht nur aus eurem Kotlin-Code. Es braucht zusätzlich:

- **Abhängigkeiten** (Libraries), z.B. Compose Multiplatform fürs UI oder JUnit für die Tests — die müssen aus dem Internet heruntergeladen werden.
- Einen **Kompilier-Schritt**, der euren Kotlin-Code in etwas übersetzt, das die Java Virtual Machine (JVM) ausführen kann.
- Einen Weg, das Ganze **zu starten** oder die **Tests laufen zu lassen**.

Ohne Gradle müsstet ihr das alles von Hand mit einzelnen Kommandozeilen-Befehlen erledigen. Gradle nimmt euch das ab: ein Befehl, und alles Nötige passiert automatisch in der richtigen Reihenfolge.

Was beim Kompilieren eigentlich passiert

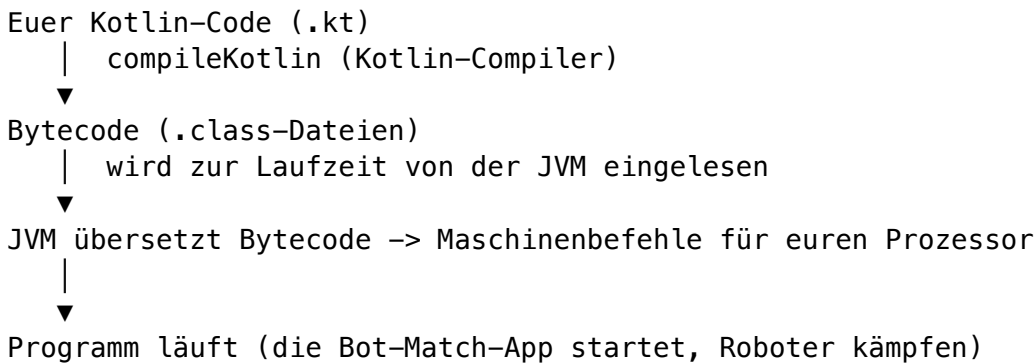
Wenn Gradle die Task `compileKotlin` ausführt, übersetzt es euren menschenlesbaren Kotlin-Code in **Bytecode** — eine kompaktere, maschinennahe Zwischensprache aus Zahlen-Codes, die kein Mensch mehr direkt lesen würde (vergleichbar mit dem, was landläufig “Binärcode” genannt wird). Diese `.class`-Dateien liegen danach z.B. unter `build/classes/`.

Das Besondere: Dieser Bytecode ist **nicht** direkt für einen bestimmten Prozessor (Intel, ARM, ...) gedacht, sondern für eine gedachte Maschine, die **Java Virtual Machine (JVM)**. Die JVM ist ein Programm, das auf eurem Betriebssystem läuft und diesen Bytecode zur Laufzeit in die tatsächlichen Maschinenbefehle für den Prozessor übersetzt, auf dem sie gerade läuft.

Warum der Umweg über die JVM statt direkt in Maschinencode für den eigenen Prozessor zu kompilieren?

- **Portabilität:** derselbe Bytecode läuft unverändert auf Windows, macOS und Linux, weil jeweils nur eine passende JVM installiert sein muss — nicht der Code selbst muss für jede Plattform neu gebaut werden (“Compile once, run anywhere”).
- **Eine JVM für mehrere Sprachen:** Kotlin, Java und einige andere Sprachen kompilieren alle zum selben Bytecode-Format. Deshalb könnt ihr in Kotlin geschriebenen Code z.B. problemlos mit Java-Bibliotheken kombinieren (genau das passiert in diesem Projekt mit JUnit, das in Java geschrieben ist).
- **Fertige Laufzeit-Umgebung:** die JVM bringt selbst schon Speicherverwaltung (Garbage Collection) und andere Grundfunktionen mit, um die ihr euch beim Programmieren nicht selbst kümmern müsst.

Kurz zusammengefasst, der Weg vom Code bis zum laufenden Bot-Match:



Das **JDK** (Java Development Kit), das ihr für dieses Projekt braucht, bringt sowohl die JVM (zum Ausführen) als auch die nötigen Compiler-Werkzeuge (zum Übersetzen) mit — deshalb reicht eine einzige Installation für beides.

Die drei wichtigsten Begriffe

1. Build-Skript (build.gradle.kts)

Die Datei, die beschreibt, **woraus** das Projekt besteht: welche Kotlin-Version, welche Abhängigkeiten (Compose, JUnit), welche Kotlin-Version fürs Compose-Plugin. Ein Auszug aus diesem Projekt:

```
dependencies {
    implementation(compose.desktop.currentOs)
    testImplementation(kotlin("test"))
    testImplementation("org.junit.jupiter:junit-jupiter:5.10.2")
}
```

Das sagt Gradle: “Lade Compose für Desktop und JUnit 5 herunter, und stelle sie dem Projekt zur Verfügung.” Ihr müsst diese Datei im Praktikum nicht anfassen.

2. Task

Eine Task ist **ein einzelner Arbeitsschritt**, den Gradle ausführen kann — z.B. “kompile den Code” oder “führe die Tests aus”. Tasks können voneinander abhängen: die Task `build` löst automatisch erst `test`, davor wiederum `compileKotlin` aus, ohne dass ihr das einzeln aufrufen müsst.

3. Gradle Wrapper (gradlew)

`./gradlew` ist ein mitgeliefertes Skript, das die **exakt richtige Gradle-Version** für dieses Projekt automatisch herunterlädt und benutzt (hier: 8.6) — ihr müsst also nicht selbst Gradle auf eurem Rechner installieren oder euch um Versionskonflikte kümmern. Deshalb ruft man im Terminal immer `./gradlew ...` auf, nie `gradle ...` direkt.

Die drei Befehle, die ihr braucht

```
./gradlew run      # startet die Compose-Desktop-App
./gradlew test     # führt die Engine-Unit-Tests aus (JUnit 5)
./gradlew build    # kompiliert alles + führt Tests aus
```

- **run** ist der Befehl, den ihr am häufigsten braucht: er startet die App, in der ihr euren Bot gegen einen Beispiel-Bot antreten lassen könnt.
- **test** führt die vorhandenen Engine-Tests aus (die testen das Framework, nicht euren Bot-Code) — praktisch, um zu prüfen, dass die Spielregeln noch wie erwartet funktionieren.

- **build** macht beides: kompiliert das gesamte Projekt und lässt danach die Tests laufen. Guter Befehl, um vor einem Testduell einmal sicherzugehen, dass alles fehlerfrei kompiliert.

Was, wenn ein Befehl fehlschlägt?

Meistens ein Kompilier-Fehler in eurem Bot-Code (z.B. Tippfehler, fehlende Klammer). Gradle zeigt dabei die Datei und Zeile an, in der der Fehler steckt — lest die Fehlermeldung von oben nach unten, die erste rot markierte Zeile ist meistens die eigentliche Ursache.

Beim allerersten `./gradlew run` (oder `build/test`) braucht Gradle zusätzlich **Internetzugang**, um die Abhängigkeiten und ggf. die passende Gradle-Version herunterzuladen — das kann beim ersten Mal ein bis zwei Minuten dauern. Danach liegt alles im lokalen Cache und spätere Aufrufe sind deutlich schneller.

In IntelliJ

Wer lieber über die IDE statt das Terminal arbeitet: IntelliJ erkennt `build.gradle.kts` automatisch beim Öffnen des Projektordners und bietet die gleichen Tasks über die Gradle-Seitenleiste an. Der grüne Play-Button an `framework/Main.kt` -> `main()` entspricht dabei `./gradlew run`.